Security Audit Report

ZIGChain v4

v1.1

May 15, 2026

Table of Contents

License

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Highend Technologies LLC to perform a security audit of the ZIGchain Cosmos chain v4.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/ZIGChain/zigchain-private
Commit	`cbcc293f6ac93d6de30c411f0193adbf857535f3`
Scope	The scope is restricted to the following folders: • `app` • `wasmbinding` • `x`
Fixes verified at commit	• `73877afd31e0403664ef2877322dbfd2e7329367` (https://github.com/ZIGChain/zigchain-private)

	• `1beec2759b759bf31fa5a3dcfd0d389e564fa3fb` (https://github.com/ZIGChain/zigchain) The git tree hash of the private repository at commit `73877af` (tag `v4.0.0`) is `0812f84f09f3a51633449dae23b8044a9dc9b828`. The public repository at commit `1beec27` (tag `v4.0.0`) resolves to the same tree hash, confirming that both repositories contain identical source trees. This can be independently verified by running `git rev-parse <commit>^{tree}` on either repository. Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

ZIGChain is a Cosmos SDK–based blockchain that provides a native ZIG token and a set of application modules for asset issuance, trading, and external bridging.

The factory module lets accounts and smart contracts create new, namespaced fungible tokens, define their minting limits, and manage metadata and admin rights over time.

The dex module implements on-chain liquidity pools for pairs of tokens, allowing users to add and remove liquidity and swap between assets using an automated market-maker model.

The tokenwrapper module connects ZIGChain with Axelar and EVM chains by locking and releasing tokens across chains, handling decimal conversions, and exposing operator/pauser controls to pause or update bridge settings when needed.

These custom modules run alongside standard Cosmos functionality for accounts, balances, staking, governance, IBC, and smart contracts, forming the main business logic surface of the chain.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	The chain implements multiple custom modules (exchange, token factory, bridge) and IBC flows.
Code readability and clarity	Medium-High	-
Level of documentation	Medium	Core modules (DEX, Factory, TokenWrapper) have solid README-style documentation and examples, but high-level system overview and inline comments are present at a moderate, not exhaustive, level.
Test coverage	Medium-High	The test coverage reported by `go test` is `60.9%`

Summary of Findings

No	Description	Severity	Status
1	Pool transfer ante handler is bypassable via authz grants and CosmWasm contract execution	Critical	Resolved
2	`RecoverZig` lacks module address and vesting account blocklists	Major	Resolved
3	DEX pool funds are sendable to blocked module addresses via the receiver parameter	Major	Resolved
4	Missing `ClaimDenomAdmin` handler in CosmWasm message plugin	Major	Resolved
5	`AddLiquidity` returns excess deposit coins to the receiver instead of the original sender	Major	Resolved
6	Denial of service on fee withdrawal via unbounded balance iteration on factory module account	Major	Resolved
7	Denial of service on liquidity removal via unbounded balance iteration on pool addresses	Major	Resolved
8	Improper error handling in `OnAcknowledgementPacket` causes irrecoverable silent fund loss	Major	Resolved
9	Migration logic disables minimal liquidity lock enabling share inflation attack	Major	Acknowledged
10	Truncation rounding in liquidity share calculation slightly favors depositors	Minor	Resolved
11	Minimum LP token requirement causes liquidity lockout for small holders in imbalanced pools	Minor	Acknowledged
12	Rate limiter middleware ordering causes incorrect rate limiting for token wrapper transfers	Minor	Resolved
13	Incomplete pool transfer blocking allows dust injection	Minor	Acknowledged
14	Compounded rounding error in liquidity addition ratio calculation	Minor	Resolved
15	Chain resume after halt without LP grace period enables stale-price arbitrage	Minor	Acknowledged

16	Hard error in `OnAcknowledgementPacket` blocks acknowledgement processing for non-whitelisted channels	Minor	Resolved
17	IBC settings changes during in-flight packet lifecycle cause incorrect refund amounts	Minor	Acknowledged
18	Factory `MintAndSendTokens` bypasses pool transfer blocking ante handler	Minor	Resolved
19	Attacker-controlled denomination sorting blocks legitimate fee withdrawal	Minor	Resolved
20	Missing rollback in `SendPacket` after `ics4Wrapper.SendPacket` failure	Minor	Resolved
21	WASM custom message handlers return `nil` for events, data, and message responses	Minor	Resolved
22	Acknowledgement refund amount is not validated against the originally locked amount	Minor	Acknowledged
23	Centralized module wallet withdrawal enabling unilateral fund extraction	Minor	Acknowledged
24	Incomplete genesis invariant validation allowing internally inconsistent module state	Minor	Resolved
25	Pool UID collision possible via separator character in denomination names	Minor	Resolved
26	Misleading denomination supply metrics due to inconsistent burn and mint accounting	Minor	Resolved
27	Over-privileged factory module account granting unnecessary staking authority	Minor	Resolved
28	Migration does not propagate admin index reconstruction failures	Minor	Acknowledged
29	Unvalidated and unused description field in denom creation message	Minor	Resolved
30	Missing stateful IBC validation allowing misconfiguration of channel and client identifiers	Minor	Acknowledged
31	Missing input validation in CosmWasm query plugin	Informational	Resolved
32	Incorrect variable reference in the `AddLiquidity` error message	Informational	Resolved
33	Duplicated `IsDexPoolAddress` logic creates maintenance risk	Informational	Resolved

34	Missing post-swap reserve validation in `SwapExactOut`	Informational	Resolved
35	Excessive maximum custom message size for WASM bindings	Informational	Resolved
36	IBC and transfer queries not whitelisted for WASM stargate queries	Informational	Resolved
37	Stargate query amplification via unbounded response marshalling	Informational	Acknowledged
38	`SwapOut` query type defined but not wired in custom querier	Informational	Resolved
39	`ValidateBasic` allows zero-amount coins that fail deeper in execution	Informational	Resolved
40	Global max slippage parameter restricts LP flexibility for volatile pairs	Informational	Acknowledged
41	Lexicographic pool ordering in query results	Informational	Acknowledged
42	All IBC settings fields lack per-packet caching causing misbehavior during in-flight packets	Informational	Acknowledged
43	Channel ordering mismatch between `TokenWrapper` IBC module tests and ICS-20 specification	Informational	Resolved
44	Channel validation on `OnTimeoutPacket` incorrectly requires `OPEN` state	Informational	Resolved
45	Factory token burn authorization is asymmetric with minting authorization	Informational	Acknowledged

Detailed Findings

1. Pool transfer ante handler is bypassable via authz grants and CosmWasm contract execution

Severity: Critical

In `x/dex/ante/block_pool_transfers.go:28-86`, the `BlockPoolTransfersDecorator` inspects top-level transaction messages and rejects `MsgSend` and `MsgMultiSend` where the recipient is a DEX pool address. This decorator is intended to ensure that pool addresses receive funds only through controlled DEX module functions, such as `CreatePool`, `AddLiquidity`, and `Swap`.

However, the decorator only inspects the outer message types. When a `MsgSend` is wrapped inside a `MsgExec` from `x/authz`, the decorator sees only the `MsgExec` envelope and does not recurse into the inner messages. The same bypass applies to CosmWasm contracts that execute bank sends via `stargate` or `bank` messages, as these are dispatched through the message router after ante handling.

Consequently, an attacker can use authz grants or deployed contracts to send arbitrary tokens directly to a pool address, breaking the constant-product invariant that the DEX module relies on.

Recommendation

We recommend extending the `BlockPoolTransfersDecorator` to recursively inspect inner messages within `MsgExec` envelopes from `x/authz`. For CosmWasm-originated transfers, we recommend adding a post-handler or middleware at the message router level that validates the recipient of any bank send dispatched by a contract.

Status: Resolved

2. `RecoverZig` lacks module address and vesting account blocklists

Severity: Major

In `x/tokenwrapper/keeper/keeper.go:567`, the `RecoverZig` function checks whether the target address matches the operator address or a DEX pool address, and rejects recovery on those addresses. This function converts IBC vouchers held by an address into native `uzig` by locking the vouchers in the tokenwrapper module and unlocking native tokens to the same address.

However, the blocklist does not cover vesting accounts and other module accounts, such as the `x/distribution` module. The `x/distribution` module account holds outstanding

validator and delegator rewards, and its reward accounting via `ValidatorOutstandingRewards` and `ValidatorCurrentRewards` is keyed per denomination. Executing `RecoverZig` on this address would swap IBC vouchers for native `uzig` under the `x/distribution` module, corrupting reward records that reference the original denomination.

Consequently, any account with the appropriate signer authority can trigger `RecoverZig` on unprotected module addresses and vesting accounts, potentially causing a denial of service.

Recommendation

We recommend extending the blocklist in `RecoverZig` to reject all module accounts.

Additionally, vesting accounts should be blocked, as converting the denomination of vesting funds would break the enforcement of the vesting schedule.

Status: Resolved

3. DEX pool funds are sendable to blocked module addresses via the receiver parameter

Severity: Major

In `x/dex/keeper/pool.go:325`, the `SendFromPoolToAddress` function uses `bankKeeper.SendCoins` to transfer tokens from a pool address to an arbitrary receiver. This function is called during `Swap`, `RemoveLiquidity`, and `AddLiquidity` refund operations.

However, `bankKeeper.SendCoins` does not enforce the blocked list validations as enforced in `app/keepers/keepers_config.go:205-216`. A user can set `msg.Receiver` to any address, including blocked module accounts such as the fee collector, distribution module, or other system accounts.

Consequently, an attacker can route swap outputs or liquidity withdrawal proceeds to module accounts that are not designed to receive arbitrary external funds. This can corrupt module-specific accounting, inflate balances used in reward calculations, or interfere with governance deposit tracking.

Recommendation

We recommend adding a blocked-address check to `SendFromPoolToAddress` before executing `bankKeeper.SendCoins` call.

Status: Resolved

4. Missing `ClaimDenomAdmin` handler in CosmWasm message plugin

Severity: Major

In `wasmbinding/message_plugin.go:94`, the custom message dispatcher routes the provided `ZMsg` message to their dedicated entry points. One of the message types is `ClaimDenomAdmin`, indicating that contracts are expected to be able to claim the denom admin role after a proposal is made.

However, the `switch` statement in the message plugin does not include the `ClaimDenomAdmin` case. When a contract sends a `ClaimDenomAdmin` message, the transaction fails with an "unsupported custom message" error.

Consequently, any CosmWasm contract that attempts to claim a denom admin role through the custom message binding will fail, preventing the protocol from working as intended.

Recommendation

We recommend adding a `case contractMsg.ClaimDenomAdmin != nil` branch and invoking the appropriate `claimDenomAdmin` handler method.

Status: Resolved

5. `AddLiquidity` returns excess deposit coins to the receiver instead of the original sender

Severity: Major

In `x/dex/keeper/msg_server_add_liquidity.go:157`, after computing the actual base and quote amounts required for the liquidity deposit, the handler sends `returnedCoins` to the `receiver` address instead of the `signer` address.

The `returnedCoins` represent the difference between what the signer deposited and what the pool actually consumed. When `msg.Receiver` differs from `msg.Signer`, the excess funds originating from the signer are sent to the receiver, effectively transferring value from the signer to an unrelated party without the signer's explicit intent.

Consequently, any transaction where the signer specifies a different receiver results in the signer losing the excess deposit amount.

Recommendation

We recommend changing the recipient of `returnedCoins` in line `157` from `receiver` to `signer`, ensuring that excess deposit funds are always returned to the account that originally provided them.

Status: Resolved

6. Denial of service on fee withdrawal via unbounded balance iteration on factory module account

Severity: Major

The `WithdrawModuleFees` function in `x/factory/keeper/msg_server_withdraw_module_fees.go:42` calls `GetAllBalances` on the factory module account address to retrieve accumulated fees. Anyone can create factory denominations by paying the creation fee, and each newly created denomination results in a balance entry on the module account.

An attacker can mint thousands of dummy factory token denominations to inflate the module account balance set.

Although the code paginates the actual `SendCoins` operation via `MaxDenomsPerWithdrawal`, this mitigation is insufficient because the `GetAllBalances` call executes before the pagination and iterates over every denomination in the KV store for that address. At `10,000` or more denominations, the gas cost of `GetAllBalances` alone exceeds the block gas limit, permanently bricking fee withdrawal.

Recommendation

We recommend replacing `GetAllBalances` with a targeted query that only retrieves denominations known to generate fees. Alternatively, maintain an explicit set of fee-generating denominations and iterate only over that set.

Consider adding the factory module account to `blockAccAddrs` to prevent unsolicited token transfers.

Status: Resolved

7. Denial of service on liquidity removal via unbounded balance iteration on pool addresses

Severity: Major

The `RemoveLiquidity` function in `x/dex/keeper/msg_server_remove_liquidity.go:128` calls `GetAllBalances` on the pool address to perform a solvency check. This function iterates over every denomination held by the pool address.

If an attacker sends thousands of dust denominations to a pool address (via pre-computed pool addresses that bypass the ante handler), the KV store reads and `sdk.Coins.Add` operations (which are `O(N)` per call due to sorted-slice search) result in `O(N^2)` total cost.

At approximately `10,000` or more denominations, the gas consumed exceeds the block gas limit, permanently bricking `RemoveLiquidity` for the targeted pool.

Recommendation

We recommend replacing the `GetAllBalances` solvency check with a targeted approach that only queries the two expected pool denominations.

Status: Resolved

8. Improper error handling in `OnAcknowledgementPacket` causes irrecoverable silent fund loss

Severity: Major

In `x/tokenwrapper/module/on_acknowledgment_packet.go:109`, the `OnAcknowledgementPacket` function calls the underlying ICS-20 handler `im.app.OnAcknowledgementPacket` but silently discards its error. The error variable is assigned in the `if` statement but the body is empty, so execution continues regardless of whether the underlying handler succeeded.

When the underlying ICS-20 acknowledgement handler fails (for example, because the sender's address was blocked by the bank module after sending, or because the module lacks sufficient balance), the standard ICS-20 refund logic does not execute. For a failed acknowledgement, the function proceeds to `handleRefund` in line `126`, which attempts to lock IBC vouchers from the sender.

However, because the underlying handler's refund did not execute, the sender does not hold the IBC vouchers that `handleRefund` expects. This causes `handleRefund` to fail at the `CheckAccountBalance` call in `x/tokenwrapper/module/handlers.go:47`.

The `handleRefund` error is caught in line `127`, logged in line `128`, and the function returns `nil` in line `130`, signaling success to the IBC core.

Consequently, the user's native uzig tokens remain permanently locked in the module (locked during `SendPacket`), the IBC vouchers burned during `SendPacket` are not restored, and no error is propagated to indicate the failure. This results in a total and silent loss of user funds.

Additionally, on the success acknowledgement path, the function does not delegate to `im.app.OnAcknowledgementPacket` after line `109`. If the underlying ICS-20 handler needs to perform bookkeeping on successful acknowledgements, this bookkeeping is skipped.

Recommendation

We recommend propagating the error from `im.app.OnAcknowledgementPacket` instead of swallowing it. If the underlying handler fails, the entire `OnAcknowledgementPacket` call should return the error so that the IBC core does not mark the acknowledgement as processed. This ensures the packet can be retried and prevents silent fund loss.

Status: Resolved

9. Migration logic disables minimal liquidity lock enabling share inflation attack

Severity: Major

The `V2Migration` function in `x/dex/keeper/k_get_set_params.go` forcefully sets `params.MinimalLiquidityLock = 0` and persists the modified parameters via `SetParams` without re-running parameter validation. This directly contradicts the module's existing validation logic, where `validateMinimalLiquidityLock` enforces that `MinimalLiquidityLock` must be non-zero during normal operation.

The `MinimalLiquidityLock` parameter serves as an economic safeguard against share inflation attacks during pool initialization. In the standard pool creation flow, `initialLiquidityShares` requires `lpAmount > minimalLock` and permanently locks a portion of the initial LP shares inside the pool account. This ensures that early liquidity providers cannot fully withdraw the seed liquidity and prevents disproportionate share minting through minimal initial deposits.

By setting `MinimalLiquidityLock` to zero during migration, the protocol effectively removes this protective mechanism. As a result, the square root–based liquidity calculation (`integerSqrt(product)`) assigns the entire minted share amount to the first depositor without reserving any locked portion. This allows pools to be created or re-created with arbitrarily small initial deposits.

This change reintroduces the classic “first depositor” attack vector observed in automated market maker (AMM) designs such as Uniswap V2. A malicious actor can initialize or reinitialize a pool with negligible liquidity, then front-run legitimate liquidity additions or price movements to mint a disproportionately large share of total LP tokens at minimal cost. As subsequent liquidity providers add capital, the attacker can extract value by redeeming their inflated LP position, resulting in economic loss for honest participants.

Recommendation

We recommend preserving the invariant that `MinimalLiquidityLock` must be strictly greater than zero, including during migration procedures.

Status: Acknowledged

10. Truncation rounding in liquidity share calculation slightly favors depositors

Severity: Minor

In `x/dex/keeper/msg_server_add_liquidity.go:293`, the `CalculateLiquidityShares` function computes `actualBase` by multiplying the quote amount by the pool ratio and calling `TruncateInt`.

In line `302`, `actualQuote` is similarly computed using truncation when the base-limited branch is taken. These truncated values determine how many tokens the pool actually consumes from the depositor.

However, truncation rounds toward zero, meaning the pool consistently accepts slightly fewer tokens than the precise ratio demands. The depositor receives LP shares calculated from the full amount of the constraining token, but the counterpart token contribution is rounded down.

Over many deposits, this systematic under-contribution dilutes existing LP holders because the pool ratio drifts from the value implied by the outstanding LP token supply. The effect is small per transaction but compounds over the lifetime of an active pool.

Recommendation

We recommend replacing `TruncateInt` with `Ceil` on lines `293` and `302` when calculating `actualBase` and `actualQuote`.

Ceiling ensures the pool demands at least the proportionally correct amount from the depositor, protecting existing LP holders from dilution.

Status: Resolved

11. Minimum LP token requirement causes liquidity lockout for small holders in imbalanced pools

Severity: Minor

In `x/dex/keeper/msg_server_remove_liquidity.go:66`, the `RemoveLiquidity` handler enforces a minimum LP token amount computed by `MinLPTokensForTwoCoinsOut` in line `62`.

This function calculates the smallest LP token quantity that guarantees that both the base and quote token outputs are at least `1` unit. A secondary enforcement is in effect in line `138`, where the handler rejects any removal that yields fewer than two output coins.

However, in highly imbalanced pools where one reserve is orders of magnitude larger than the other, the minimum LP token threshold can become prohibitively high. A liquidity provider holding fewer LP tokens than this minimum may be unable to withdraw any liquidity, even though redeeming their tokens for only the dominant-side coin would be mathematically valid and economically reasonable. The requirement to produce both coins in every withdrawal creates an artificial lockout condition.

Consequently, small LP holders in imbalanced pools have their funds permanently locked with no recourse. This risk increases over time as pools naturally become imbalanced through trading activity, and disproportionately affects retail participants with smaller positions.

Recommendation

We recommend allowing partial withdrawals that return only one coin when the LP token amount is below the two-coin minimum. The handler should proceed with the withdrawal and return whichever coins have a non-zero output amount, rather than rejecting the entire transaction.

Additionally, we recommend implementing user-specified minimum output amounts (slippage parameters) for each coin, giving liquidity providers explicit control over the acceptable withdrawal terms.

Status: Acknowledged

12. Rate limiter middleware ordering causes incorrect rate limiting for token wrapper transfers

Severity: Minor

The rate limiter middleware is stacked before the `TokenWrapper` in the `ICS4Wrapper` chain in `app/ibc.go:229`.

Since the `TokenWrapper` scales up amounts on send (for example, from 6 to 18 decimals), the rate limiter sees the pre-scaled (smaller) amount on outgoing transfers. On incoming transfers (`OnRecvPacket`), the rate limiter sees the raw IBC packet amount (scaled-up, 18 decimals) before the `TokenWrapper` scales it down. This asymmetry means the rate limiter evaluates different magnitudes for the same logical transfer depending on direction.

We classify this issue as minor since the rate limiter can be configured to account for the scaled amounts, but the current middleware ordering creates an operational footgun where rate limits may be ineffective or overly restrictive depending on the direction of transfer.

Recommendation

We recommend reordering the middleware stack so that the rate limiter is positioned after the `TokenWrapper` in the send direction, or configure rate limits to account for the decimal scaling factor.

Status: Resolved

13. Incomplete pool transfer blocking allows dust injection

Severity: Minor

The `BlockPoolTransfersDecorator` ante handler in `x/dex/ante/block_pool_transfers.go:61` only blocks transfers to currently existing pool addresses by checking `MsgSend` and `MsgMultiSend` message types.

Pools created in the same transaction are not protected.

An attacker can pre-compute the pool address and send dust tokens to the predicted pool address before the ante handler recognizes it.

Recommendation

We recommend documenting this behaviour.

Status: Acknowledged

14. Compounded rounding error in liquidity addition ratio calculation

Severity: Minor

The `AddLiquidity` function in `x/dex/keeper/msg_server_add_liquidity.go:307` computes the pool ratio as

`poolBase.ToLegacyDec().Quo(poolQuote.ToLegacyDec())`, which performs a decimal division that rounds down. This rounded ratio is then multiplied by a token amount and truncated again. The double rounding (first in the ratio computation, then in the multiplication) compounds the error in favor of the user.

Uniswap V2 avoids this by computing the optimal counter-amount directly from reserves using a single integer operation: `amountBOptimal = amountADesired * reserveB / reserveA`. This "multiply first, then divide" pattern preserves more precision.

We classify this issue as minor since the slippage check in liquidity addition limits the practical impact, but the rounding consistently favors one direction and accumulates over many operations.

Recommendation

We recommend replacing the two-step ratio computation with a single muldiv operation: `actualBase = quote.Amount * poolBase / poolQuote` and `actualQuote = base.Amount * poolQuote / poolBase`.

Status: Resolved

15. Chain resume after halt without LP grace period enables stale-price arbitrage

Severity: Minor

If the chain halts and resumes without providing a grace period for liquidity providers to readjust their positions, stale pool prices create massive arbitrage opportunities. Arbitrageurs can drain value from LPs at outdated prices before the market re-equilibrates, as observed in the `RemoveLiquidity` and swap functions in the dex module.

The issue is compounded by the fact that liquidity addition is constrained by a fixed, governance-set max slippage parameter, which prevents LPs from quickly re-depositing at the new market price if it has moved significantly.

We classify this issue as minor since it requires an external chain halt event and is a design-level concern rather than a code bug.

Recommendation

We recommend implement a grace period mechanism after chain resumption during which swaps are temporarily disabled or restricted, allowing LPs to adjust positions. Additionally, consider making the slippage parameter user-configurable per transaction.

Status: Acknowledged

16. Hard error in `OnAcknowledgementPacket` blocks acknowledgement processing for non-whitelisted channels

Severity: Minor

The `OnAcknowledgementPacket` function in `x/tokenwrapper/module/on_acknowledgment_packet.go:31` returns a hard error when channel validation fails, rather than passing through to the underlying ICS-20 handler.

The standard ICS-20 refund logic (`refundPacketTokens`) does not require the channel to be in the `OPEN` state. If ibc-go relaxes channel state requirements for acknowledgements (as it already did for `OnRecvPacket` to allow `FLUSHING` and `FLUSHCOMPLETE` states per the ICS-04 specification), this hard return would permanently block acknowledgement processing and lock user funds.

The ICS-04 specification states: `abortTransactionUnless(channel.state === OPEN || channel.state === FLUSHING)`.

Recommendation

We recommend passing through to the underlying `im.app.OnAcknowledgementPacket` handler instead of returning a hard error. This matches the pattern used in lines `45`, `61`, and `80` in the same file for other non-matching conditions.

Status: Resolved

17. IBC settings changes during in-flight packet lifecycle cause incorrect refund amounts

Severity: Minor

The `UpdateIbcSettings` message in `x/tokenwrapper/keeper/msg_server_update_ibc_settings.go:21` allows the

operator to change `decimalDifference`, `nativeChannel`, `nativePort`, and other IBC parameters at any time.

The `SendPacket` function scales up amounts using the current `decimalDifference`, and the scaled amount is stored in the packet data. When the acknowledgement or timeout arrives (potentially hours or days later), `ScaleDownTokenPrecision` uses the current `decimalDifference`, which may have changed.

Since settings are not cached per packet, updating `decimalDifference` after send but before acknowledgement can either create more `uzig` on acknowledgement failure (if the difference is reduced) or lose user funds (if the difference is increased).

We classify this issue as minor since it requires operator action, but the consequences for in-flight packets are severe.

Recommendation

We recommend storing the `decimalDifference` (and other relevant settings) per packet sequence at send time.

On acknowledgement or timeout, use the stored settings rather than the current ones. Alternatively, store the latest packet sequence for send packets and if the acknowledgement returns after IBC setting updates, use the previous settings.

Status: Acknowledged

18. Factory `MintAndSendTokens` bypasses pool transfer blocking ante handler

Severity: Minor

The `MintAndSendTokens` function in `x/factory/keeper/msg_server_mint_and_send_tokens.go:52` accepts an arbitrary `Recipient` address.

The `BlockPoolTransfersDecorator` ante handler blocks `MsgSend` to pool addresses, but `MintAndSendTokens` uses `bankKeeper.SendCoinsFromModuleToAccount`, which bypasses the ante handler entirely.

We classify this issue as minor since it requires the attacker to be the bank admin of the same factory denomination.

Recommendation

We recommend adding a pool address check within the `MintAndSendTokens` handler that validates the recipient is not a known pool address before executing the mint.

Status: Resolved

19. Attacker-controlled denomination sorting blocks legitimate fee withdrawal

Severity: Minor

The `WithdrawModuleFees` function in `x/factory/keeper/msg_server_withdraw_module_fees.go:85` truncates the balance list at `MaxDenomsPerWithdrawal` using index-based slicing. If an attacker can stuff `100` or more denominations that sort lexicographically before the legitimate fee denominations, every withdrawal call only withdraws attacker dust and never reaches the real fees.

We classify this issue as minor since it depends on the attacker paying the creation fee for each denomination, but the cost may be low relative to the fees being blocked.

Recommendation

We recommend, rather than relying on lexicographic ordering, maintaining an explicit allowlist of fee-generating denominations or query balances for specific known denominations instead of using `GetAllBalances`.

Status: Resolved

20. Missing rollback in `SendPacket` after `ics4Wrapper.SendPacket` failure

Severity: Minor

In the `SendPacket` function in `x/tokenwrapper/module/send_packet.go:241`, after native tokens are locked (line `211`) and IBC vouchers are burned (line `221`), the function calls `w.ics4Wrapper.SendPacket` in line `235`.

If this call fails, the error is returned without rolling back the lock or burn operations. This is inconsistent with the burn failure path in lines `222–229`, which correctly rolls back the lock. It

is a code inconsistency only, as the Cosmos SDK cache context should revert the entire transaction on error propagation.

Recommendation

We recommend adding an explicit rollback logic for the `ics4Wrapper.SendPacket` failure path, matching the pattern used for burn failure: unlock the previously locked native tokens and mint back the burned IBC vouchers.

Status: Resolved

21. WASM custom message handlers return `nil` for events, data, and message responses

Severity: Minor

All custom message handlers in the WASM message plugin in `wasmbinding/message_plugin.go:538` return `nil, nil, nil, nil` on success.

If a contract uses submessages with reply callbacks, this return signature silently drops the reply data, breaking the submessage and reply flow.

For example, when a contract calls pool creation, it would expect the pool ID and pool address to be stored in state in the reply handler, but receives empty data instead.

Events emitted by the underlying keeper are added to the context event manager but are not returned via the WASM dispatch return values.

Recommendation

We recommend returning the appropriate events, data, and `msgResponses` from each custom message handler, matching the pattern used by the default `SDKMessageHandler` in wasmd.

Status: Resolved

22. Acknowledgement refund amount is not validated against the originally locked amount

Severity: Minor

In `x/tokenwrapper/module/on_acknowledgment_packet.go:92`, the `OnAcknowledgementPacket` function parses the refund amount from the IBC packet data

using `sdkmath.NewIntFromString(data.Amount)`. This amount originates from the counterparty chain's acknowledgement and is used directly in the refund and tracking logic without validation against the amount that was actually locked during `SendPacket`.

During `SendPacket` in `x/tokenwrapper/module/send_packet.go:211`, the module locks the sender's native uzig tokens and burns IBC vouchers based on the original transfer amount.

However, the acknowledgement handler trusts the amount field from the packet data, which could differ from the locked amount if the counterparty chain returns corrupted or manipulated data. The underlying `im.app.OnAcknowledgementPacket` handler may validate this, but its errors are silently discarded in line `109`.

Consequently, the `handleRefund` function in line `126` and the `ScaleDownTokenPrecision` call in line `141` both operate on an unverified value, potentially unlocking more or fewer `uzig` tokens than were originally locked. Since `uzig` is native to the zig chain, the zig chain should only trust values that originates from it.

Recommendation

We recommend storing the original pre-scaled amount per packet sequence at send time and using the stored value for refund calculations in `OnAcknowledgementPacket` and `OnTimeoutPacket`, rather than trusting the amount from the packet data. The `uzig` refund logic should only use values originating from ZIGChain.

Status: Acknowledged

23. Centralized module wallet withdrawal enabling unilateral fund extraction

Severity: Minor

The `WithdrawFromModuleWallet` function allows the current operator to withdraw arbitrary amounts of tokens from the tokenwrapper module account, provided that the `msg.Signer` matches the stored operator address. No additional safeguards such as governance approval, multi-signature enforcement, timelocks, rate limiting, withdrawal caps, or destination restrictions are implemented.

This design introduces a strong centralization assumption in which a single privileged key has unilateral control over treasury-level funds. Since the module wallet backs bridge solvency and provides native liquidity for redemptions and recovery operations, compromise or misuse of the operator key could result in complete drainage of funds. Such an event could break peg mechanisms, disrupt redemption guarantees, and impose financial losses or forced recovery delays on users.

While this behavior may be intentional for operational flexibility, it creates a single point of failure. From a threat modeling perspective, compromise of the operator key would directly enable unrestricted treasury extraction, significantly impacting protocol integrity and user trust.

Recommendation

We recommend reducing the centralization risk associated with module wallet withdrawals by introducing stronger authorization controls. Withdrawal authority should ideally be placed under governance control or, at minimum, enforced through a multisig-controlled operator account combined with a timelock mechanism.

Status: Acknowledged

24. Incomplete genesis invariant validation allowing internally inconsistent module state

Severity: Minor

The `GenesisState.Validate` function in `x/tokenwrapper/types/genesis.go` verifies that each genesis field is well-formed in isolation, but it omits relational and cross-field invariant checks. This allows a genesis file to pass validation even when its overall state is collectively inconsistent with the module's intended runtime constraints.

In particular, `PauserAddresses` entries are individually validated as `bech32` addresses, but the list is not checked for duplicates. A hand-crafted or imported genesis can therefore include the same pauser address multiple times, inflating the stored list and creating unnecessary state bloat. This can also introduce avoidable overhead during runtime checks such as `IsPauserAddress`, which may iterate redundant entries and waste gas/compute without changing authorization semantics.

Additionally, `OperatorAddress` and `ProposedOperatorAddress` are validated only for address correctness and are never compared against each other. This permits a genesis state where both fields are identical, despite the module's runtime proposal logic explicitly rejecting the "same address" case. While this is unlikely to be directly exploitable, it creates a genesis configuration that the module's own operational rules treat as invalid, which is misleading for operators and auditors and may complicate incident response or migrations.

Finally, the counters `TotalTransferredIn` and `TotalTransferredOut` are only checked for non-negativity, with no invariant establishing that their relationship is coherent with expected bridge accounting assumptions. This can mislead off-chain monitoring, solvency dashboards, and queries that interpret these counters as reliable flow indicators, potentially obscuring abnormal conditions or producing incorrect analytics.

Recommendation

We recommend extending `GenesisState.Validate` with explicit cross-field and relational invariant checks.

Status: Resolved

25. Pool UID collision possible via separator character in denomination names

Severity: Minor

Pool UIDs are constructed in `x/dex/keeper/pool_uids.go:23` by joining sorted denomination names with a "`-`" separator.

Denomination names containing the "`-`" character can cause UID collisions.

For example, a pool with denominations (`"usdt-wrapped"`, `"eth"`) and a pool with denominations (`"usdt"`, `"wrapped-eth"`) would both produce the UID `"usdt-wrapped-eth"`.

Recommendation

We recommend using a separator character that is not permitted in denomination names, or use a different UID construction method such as hashing the sorted denomination pair.

Status: Resolved

26. Misleading denomination supply metrics due to inconsistent burn and mint accounting

Severity: Minor

The `calculateDenomStats` query helper in `x/factory/keeper/query_denom_all.go` derives `totalBurned` and `maxSupply` values in a manner that is inconsistent with the module's minting and burning semantics.

At the state level, `denom.Minted` functions as a lifetime mint counter that is monotonically increasing. It is incremented during `MintAndSendTokens` execution and is never reduced when tokens are burned. The minting logic enforces `Minted <= MintingCap` as a strict lifetime cap, meaning that once the cumulative minted amount reaches `MintingCap`, no further minting is permitted, regardless of the current circulating supply.

However, the query logic computes `totalBurned` as `Minted - currentSupply`, and then derives `maxSupply` as `MintingCap - totalBurned`. This calculation implies that burned tokens restore minting capacity and that supply can grow again up to the reported maxSupply. In practice, this is incorrect: burned tokens do not decrease `Minted`, and therefore do not increase remaining mintable capacity.

As a result, the `total_burned` and `max_supply` values exposed through the gRPC/REST API provide a misleading representation of future minting capability. Off-chain dashboards, monitoring systems, and integrators relying on these fields may incorrectly assume that burning tokens reopens minting headroom, potentially leading to inaccurate economic modeling or supply projections.

Recommendation

We recommend aligning the query-layer calculations with the module's actual minting invariants.

Status: Resolved

27. Over-privileged factory module account granting unnecessary staking authority

Severity: Minor

In `app/keepers/keepers_config.go`, the factory module account is configured with `Minter`, `Burner`, and `Staking` permissions.

However, the factory module does not perform any staking-related operations such as delegation, undelegation, validator interaction, or reward management. Its functional requirements are fully satisfied by the `Minter` and `Burner` capabilities.

Granting the additional `Staking` permission violates the principle of least privilege.

Recommendation

We recommend removing the `authtypes.Staking` permission from the factory module account configuration.

Status: Resolved

28. Migration does not propagate admin index reconstruction failures

Severity: Minor

The `V2Migration` function in `x/factory/keeper/k_set_denom.go` migrates legacy denom structures and subsequently calls `MigrateAdminDenomAuthList` to rebuild the admin-to-denom index. However, the migration logic does not propagate or handle errors that may occur during the admin index reconstruction phase.

If `MigrateAdminDenomAuthList` encounters an issue, such as state iteration errors, malformed entries, or decoding failures, the migration process will still return nil and appear to have completed successfully.

This creates a risk of a partially migrated state where canonical `DenomAuth` records exist, but the derived admin index is incomplete, inconsistent, or stale.

Recommendation

We recommend modifying `V2Migration` to properly handle and propagate errors from `MigrateAdminDenomAuthList`.

Status: Acknowledged

29. Unvalidated and unused description field in denom creation message

Severity: Minor

The `MsgCreateDenom` message in `x/factory/types/msg_create_denom.go` includes a `Description` field that is neither validated in `ValidateBasic` nor persisted during denom creation. While the function enforces strict validation rules for `Creator`, `SubDenom`, `MintingCap`, `URI`, and `URIHash`, it performs no checks on `msg.Description`.

Furthermore, during the keeper execution path for denom creation, the bank metadata is initialized with `Description: ""` instead of using `msg.Description`, effectively discarding any user-provided input. As a result, transactions may include arbitrary description data that increases transaction size and consumes network and mempool resources without producing any on-chain effect.

This creates inconsistent and misleading behavior at the interface level. Clients may reasonably expect that providing a description during denom creation will persist metadata on-chain, yet the value is silently ignored.

Recommendation

We recommend introducing explicit validation for the `Description` field in `MsgCreateDenom.ValidateBasic`, including bounded length checks to prevent excessive input size.

Status: Resolved

30. Missing stateful IBC validation allowing misconfiguration of channel and client identifiers

Severity: Minor

The `MsgUpdateIbcSettings` handler in `x/tokenwrapper/keeper/msg_server_update_ibc_settings.go` performs only syntactic validation and operator authorization before persisting updated IBC configuration values. While `ValidateBasic` ensures that fields such as client IDs, channel IDs, and ports match expected string formats, and the handler verifies that the signer matches the current operator, no stateful validation is performed against the actual on-chain IBC state.

Specifically, the handler does not verify that the provided `NativeChannel` or `CounterpartyChannel` exists, is in `OPEN` state, or is associated with the expected connection. It does not confirm that the specified `NativeClientId` or `CounterpartyClientId` corresponds to an existing light client. Nor does it check that the referenced channel and client identifiers are logically connected via an active IBC connection.

As a result, the operator can set syntactically valid but non-existent, closed, or stale identifiers and the module will accept and persist these values without error. This can silently break bridge functionality, halt cross-chain transfers, or cause redemptions to fail due to misconfigured routing.

Recommendation

We recommend introducing comprehensive stateful validation before persisting new IBC settings.

Status: Acknowledged

31. Missing input validation in CosmWasm query plugin

Severity: Informational

In `wasmbinding/query_plugin.go:83`, the `Denom` query branch reads the `denom` string directly from the contract query input and passes it to `factoryKeeper.GetDenom` without any sanitization or format validation.

Similarly, in line `134`, the `Pool` query branch reads `poolID` and passes it directly to `dexKeeper.GetPool` without calling `validators.CheckPoolId`.

The factory and DEX modules define validation functions, such as `validators.CheckDenomString` and `validators.CheckPoolId` that enforces format constraints on these identifiers. Implementing these validations is considered best practice and could benefit the query plugin by ensuring consistent behavior with the native message handlers.

Recommendation

We recommend adding calls to `validators.CheckDenomString` before line `85` and `validators.CheckPoolId` before line `136` and return a descriptive error to the contract if validation fails.

Status: Resolved

32. Incorrect variable reference in the `AddLiquidity` error message

Severity: Informational

In `x/dex/keeper/msg_server_add_liquidity.go:113`, the error message for a failed `SendFromAddressToPool` call formats the coin amount using `actualCoins.String()`. The `actualCoins` variable in line `107` contains the computed actual base and quote amounts that the pool consumes, which is a subset of what is sent.

However, the `SendFromAddressToPool` call in line `109` sends `sortedCoins`, which is the full user-supplied deposit amount, not `actualCoins`. When this send fails due to insufficient funds, the error message reports `actualCoins` instead of `sortedCoins`, displaying a different amount than what the failed operation attempted to transfer.

Recommendation

We recommend changing the error format argument in line `113` from `actualCoins.String()` to `sortedCoins.String()`.

Status: Resolved

33. Duplicated `IsDexPoolAddress` logic creates maintenance risk

Severity: Informational

In `x/tokenwrapper/keeper/keeper.go:517`, the `IsDexPoolAddress` method reimplements the pool address detection logic that already exists in `x/dex/types/pool_address.go:20` as the `IsPoolAddress` function.

Both implementations check whether an address corresponds to a module account whose name matches the pool prefix followed by a numeric suffix.

Maintaining two independent copies of the same validation logic creates a risk that future changes to pool address format or naming conventions are applied to one copy but not the other. If the implementations diverge, the tokenwrapper module may incorrectly classify addresses.

Recommendation

We recommend removing the `IsDexPoolAddress` method from the `x/tokenwrapper` keeper and instead importing and using the `IsPoolAddress` function from `x/dex/types`.

Alternatively, we recommend extracting the pool address validation into a shared utility package that both modules import.

Status: Resolved

34. Missing post-swap reserve validation in `SwapExactOut`

Severity: Informational

In `x/dex/keeper/msg_server_swap_exact_out.go:110-112`, the `SwapExactOut` handler updates the pool balances by adding the incoming amount and subtracting the outgoing amount, then persists the pool state via `SetPool`. Unlike the equivalent logic in `x/dex/keeper/msg_server_swap_exact_in.go:136-145`, this code path does not validate that both pool reserves remain strictly positive after the swap.

Without this validation, a carefully crafted `SwapExactOut` message can drain one side of the pool to zero or near-zero. The `SwapExactIn` handler explicitly iterates over pool coins and rejects any swap that would result in a zero or negative reserve, but this guard is entirely absent from `SwapExactOut`.

Recommendation

We recommend adding the same post-swap reserve validation present in `SwapExactIn` to the `SwapExactOut` handler, immediately after lines `110-111` and before `SetPool` in line `112`.

The validation should iterate over all pool coins and reject the transaction if any reserve is zero or negative, consistent with the pattern in `msg_server_swap_exact_in.go:136-145`.

Status: Resolved

35. Excessive maximum custom message size for WASM bindings

Severity: Informational

The `MaxCustomMessageSize` constant in `wasmbinding/validation.go:10` is set to `1 MB` (`1,048,576` bytes).

This is excessively large for custom messages. Only denom metadata messages might utilize a significant portion of this size, and even for metadata there is no validation on the denom units array or URI fields.

Recommendation

We recommend reducing `MaxCustomMessageSize` to a more reasonable value (for example, `64 KB`) and add field-level validation for denom metadata content.

Status: Resolved

36. IBC and transfer queries not whitelisted for WASM stargate queries

Severity: Informational

IBC and transfer custom queries (such as `EscrowAddress` and `DenomTrace`) are not included in the WASM stargate query whitelist in `wasmbinding/stargate_whitelist.go:72`.

Contracts that need to verify IBC escrow state or perform cross-chain accounting cannot query this information on-chain.

Recommendation

We recommend evaluating which IBC and transfer queries are safe to expose and add them to the stargate whitelist.

Status: Resolved

37. Stargate query amplification via unbounded response marshalling

Severity: Informational

Stargate query responses in `wasmbinding/query_plugin.go:225` are proto-decoded and then re-encoded to JSON with no depth or size checks.

A bloated response (for example, from the `DenomMetadata` query, which returns non-paginated results) could trigger expensive JSON marshalling that is not proportionally gas-metered, enabling cheap query amplification attacks from WASM contracts.

Recommendation

We recommend capping the maximum response size for stargate queries before performing JSON marshalling. Consider adding gas metering proportional to the response size.

Status: Acknowledged

38. `SwapOut` query type defined but not wired in custom querier

Severity: Informational

The `SwapOut` query type is defined in `wasmbinding/bindings/query.go:68` but is never wired into the custom querier.

WASM contracts cannot execute swap-out simulation queries despite the type being declared, making it dead code.

Recommendation

We recommend either wiring the `SwapOut` query type into the custom querier or removing the unused type definition.

Status: Resolved

39. `ValidateBasic` allows zero-amount coins that fail deeper in execution

Severity: Informational

The `ValidateBasic` function in `x/dex/types/message_add_liquidity.go:46` allows zero-amount coins, which then fail deeper in execution during the actual liquidity calculation.

This wastes gas on validation, state reads, and setup before the inevitable revert. Zero coins should be rejected upfront.

Recommendation

We recommend rejecting zero-amount coins in `ValidateBasic` to fail early and provide a clear error message.

Status: Resolved

40. Global max slippage parameter restricts LP flexibility for volatile pairs

Severity: Informational

The max slippage parameter in `x/dex/keeper/msg_server_add_liquidity.go:237` is defined as a governance-controlled global setting and is not configurable on a per-transaction basis. As a result, all liquidity providers across all pools must adhere to the same slippage tolerance threshold when adding liquidity.

This design constrains liquidity provision to price ratios that fall within a fixed deviation from the current pool price. While this may provide protective guardrails in stable market conditions, it becomes overly restrictive for volatile trading pairs where rapid price movements are common. In such cases, liquidity providers may require significantly higher slippage tolerance, potentially 10% or more, to successfully add liquidity without repeated transaction failures.

Because the parameter applies uniformly across all pool types, including pairs with different volatility profiles and liquidity depths, it introduces unnecessary rigidity into the system. This can degrade user experience, reduce capital efficiency, and discourage liquidity participation in higher-volatility markets.

Recommendation

We recommend allowing users to specify a per-transaction slippage tolerance in the `MsgAddLiquidity` message, using the governance parameter as an upper bound.

Status: Acknowledged

41. Lexicographic pool ordering in query results

Severity: Informational

Pool iteration in `x/dex/keeper/query_pool.go:40` uses a KV store prefix scan, which returns keys in lexicographic order.

Pool IDs such as `"zp1"`, `"zp10"`, `"zp11"`, `"zp2"` are ordered lexicographically rather than numerically. Any API consumer or pagination logic assuming numeric ordering will return pools in the wrong order.

Recommendation

We recommend using zero-padded numeric pool IDs (for example, `"zp0001"`, `"zp0002"`) to ensure lexicographic and numeric ordering are consistent, or document the ordering behavior for API consumers.

Status: Acknowledged

42. All IBC settings fields lack per-packet caching causing misbehavior during in-flight packets

Severity: Informational

Every setting field change in `x/tokenwrapper/keeper/msg_server_update_ibc_settings.go:21` can lead to misbehavior during in-flight packets.

This applies not only to `decimalDifference`, but also to `nativeChannel`, `nativePort`, `counterpartyPort`, `counterpartyChannel`, and other fields.

Changing any of these between send and acknowledgement or timeout causes the `TokenWrapper` to evaluate the returning packet against settings that differ from those used when the packet was created.

Recommendation

We recommend implementing per-packet settings caching or a migration lockout period that prevents any IBC settings changes while packets are in flight.

Status: Acknowledged

43. Channel ordering mismatch between `TokenWrapper` IBC module tests and ICS-20 specification

Severity: Informational

The `TokenWrapper` IBC module tests in `x/tokenwrapper/module/ibc_module_test.go:73` configure channels as `ORDERED`, but the ICS-20 transfer module requires channels to be `UNORDERED` and rejects ordered channels in `OnChanOpenInit`.

The tests only pass because they mock the underlying application. This indicates a disconnect between the test setup and actual production behavior.

Recommendation

We recommend updateing the test channel configuration to use UNORDERED channels, consistent with the ICS-20 specification, and removing the mocking that masks this mismatch.

Status: Resolved

44. Channel validation on `OnTimeoutPacket` incorrectly requires `OPEN` state

Severity: Informational

The `validateChannel` check in `x/tokenwrapper/module/on_timeout_packet.go:28` requires the channel to be in the `OPEN` state for timeout processing.

IBC core does not require the `OPEN` state for timeout processing, and neither does ICS-20 transfer.

Packets that timed out must still be resolvable regardless of the current channel state. If the channel is not `OPEN`, the user has to rely on `RecoverZig` instead of the standard timeout refund path.

Recommendation

We recommend removing the `OPEN`-only channel state check from `OnTimeoutPacket`. Timeout processing should succeed regardless of channel state to ensure user funds are always recoverable.

Status: Resolved

45. Factory token burn authorization is asymmetric with minting authorization

Severity: Informational

In `x/factory/keeper/msg_server_burn_tokens.go:67`, the `BurnTokens` function allows any token holder to burn their own factory tokens. The function verifies that the signer holds sufficient balance in line `47` and that the denomination exists in line `34`, but it does not call `k.Auth` to check whether the signer has the appropriate role.

By contrast, the `MintAndSendTokens` function in `x/factory/keeper/msg_server_mint_and_send_tokens.go:34` explicitly calls `k.Auth(ctx, msg.Token.Denom, "bank", msg.Signer)` to verify admin authorization before minting.

A denomination creator who intends to control the total supply has no mechanism to prevent holders from burning tokens. This asymmetry between minting authorization (admin-only) and burning authorization (any holder) may break tokenomics or accounting systems that assume supply changes only via admin actions.

Recommendation

We recommend documenting whether permissionless burning is the intended design. If the denomination admin should control burns, add a `k.Auth` check in `BurnTokens` consistent with the pattern in `MintAndSendTokens`. If permissionless burning is intentional, document this behavior in the module specification.

Status: Acknowledged